based on a threshold of shared functions, and I also wanted to provide users with the capacity to click each example to get more details, so I couldn't draw the network using canvas. Instead, I needed to draw the network using the same mixed-rendering method we looked at to draw all those triangles on a map. But in this case I used canvas for the network edges and SVG for the network nodes.

Using bl.ocks.org

Although D3 is suitable for building large, complex interactive applications, you often make a smal, single-use interactive data visualization that can live on a single page with limited resources. For these small applications, it's common in the D3 community to host the code on gist.github.com, which is the part of GitHub designed for small applications. If you host your D3 code as a gist, and it's formatted to have an index.html, then you can use bl.ocks.org to share your work with others.

To make your gist work on bl.ocks.org, you need to have the data files and libraries hosted in the gist or accessible through it. Then you can take the alphanumeric identifier of your gist and append it to bl.ocks.org/username/ to serve a working copy for sharing. So, for instance, I have a gist at https://gist.github.com/emeeks/0a4d7cd56e027023bf78 that demonstrates how to do the mixed rendering of a force-directed layout like I described in this chapter. As a result, I can point people to http://bl.ocks.org/emeeks/0a4d7cd56e027023bf78 and they can see the code itself as well as the animated network in action.

Doing this kind of mixed rendering with networks isn't as easy as it is with maps. That's because there's no built-in method to render regular data to canvas as with `d3.geo.path`. If you want to create a similar large network that combines canvas and SVG rendering, you have to build the function manually. First, though, you need data. This time, instead of sample geodata, listing 11.10 shows how to create sample network data.

Building sample network data is easy: you can create an array of nodes and an array of random links between those nodes. But building a sample network that's not an undifferentiated mass is a little bit harder. In listing 11.10 you can see my slightly sophisticated network generator. It operates on the principle that a few nodes are very popular and most nodes aren't (we've known about this principle of networks since grade school). This does a decent job of creating a network with 3000 nodes and 1000 edges that doesn't look quite like a giant hairball.

Listing 11.10 Generating random network data

```
var linkScale = d3.scale.linear()
        .domain([0,.9,.95,1]).range([0,10,100,1000]);

var sampleNodes = d3.range(3000).map(function(d) {
  var datapoint = {};
  datapoint.id = "Sample Node " + d;
  return datapoint;
})
```

This scale makes 90% of the links to 1% of the nodes.